COM661 Full Stack Strategies and Development

CW1 Assessment Plan

Application: LocalLens — Local Business and Review Directory

Student: YuTong Li

Student ID:202315030412

Module: COM661

Application

LocalLens will be a searchable directory for discovering independent cafés, restaurants and other local businesses, with user-submitted reviews. It addresses the practical need to compare nearby places by category, location and customer feedback. The back end will be a Python Flask API connected to MongoDB. The project will demonstrate usable resource-based CRUD, not just a database wrapper: users can browse and filter businesses and their reviews, while authenticated users can contribute reviews and administrators can moderate directory content.

Dataset

The proposed source is the Yelp Open Dataset, available for academic/non-commercial use subject to Yelp's current terms: Yelp Open Dataset (https://www.yelp.com/dataset). The source contains business and user-contributed review records, making it suitable for a meaningful parent–child model and for demonstrating search, pagination and sorting. Before implementation, I will confirm the dataset's access conditions and select a manageable local subset; I will not redistribute source data except as permitted by those conditions.

The data will be adapted into two related MongoDB collections rather than flattened into a single record:

- **businesses** (parent): `_id`, source business ID, name, address, city, postcode, coordinates, categories, opening-hours summary, average rating and review count.
- **reviews** (child): `_id`, source review ID, `business_id` reference, rating, text, date and a pseudonymous author/source-user reference.

`reviews.business_id` will reference `businesses._id`. This keeps reviews independently pageable and searchable, while allowing a business detail response to include a selected review page and computed summary. I will remove unnecessary personal fields, retain provenance where appropriate, validate references, and check for duplicates and missing parent records during preparation. The subset will be large enough to make filters and pagination meaningful; the final record counts will be reported after preparation rather than guessed in advance. MongoDB indexes will be planned for business name/location/category, business reference and review date/rating.

Functionality

The API will use noun-based resource URLs and HTTP methods consistently. Planned operations include:

- `GET /api/businesses` — list businesses with page/limit pagination, text/category/city filters, and allow-listed sort fields/directions.
- `GET /api/businesses/{business_id}` — retrieve one business and its review summary.
- `POST /api/businesses` — create a business (authenticated user; subject to the chosen permission policy).
- `PUT /api/businesses/{business_id}` and `DELETE /api/businesses/{business_id}` — update or remove a business (administrator only).
- `GET /api/businesses/{business_id}/reviews` — list that business's reviews, with pagination and rating/date filters.
- `POST /api/businesses/{business_id}/reviews` — add a review (authenticated user).
- `GET /api/reviews/{review_id}`, `PUT /api/reviews/{review_id}` and `DELETE /api/reviews/{review_id}` — retrieve, edit or remove a review; an author may manage their own review and an administrator may moderate any review.
- `POST /api/users` and `POST /api/sessions` — register and authenticate accounts; protected routes will check the authenticated user and role.

Responses will use suitable HTTP status codes (including `200`, `201`, `204`, `400`, `401`, `403`, `404` and `409` where applicable) and consistent JSON error messages. Input validation, duplicate handling, ownership checks and permission checks will be included. Flask Blueprints will separate authentication, businesses and reviews. Postman will be used to document the API and build a runnable collection with positive, validation, not-found and access-control cases; the final submission will include the collection and an actual test-run result. The final package will also include code and per-collection `mongoexport` JSON files in separate ZIPs, the required ≤5-minute demonstration video, and PDFs containing the code listing, endpoint summary, Postman results, generated API documentation and a candid self-evaluation. I will retain relevant AI chat history and explain my own design and code decisions in the walkthrough, as required by the specification.

Initial implementation sequence

1. Confirm dataset terms, acquire and profile a suitable subset, then clean and transform parent/child records.
2. Define MongoDB collections, references, indexes and validation rules; verify import/export.
3. Implement and manually exercise Flask API routes, authentication and role/ownership permissions.
4. Build the Postman collection and documentation; run the full collection and fix failures.

5. Prepare the code/data ZIPs, required PDF evidence and a concise walkthrough video; check all files against the submission checklist.